

Assignment 2 Extra

File Handling in SV

Design Specs:

The module implements a simple combinational processor that performs one of four operations on an 8-bit input data word based on a 2-bit operation select input.

Port Name	Port Type	Width	Description
data_in	Input	8 bits	Input data word to be processed.
op_sel	Input	2 bits	Operation select signal to choose the required operation.
data_out	Output	8 bits	Result of the selected operation applied to data_in.

Output changes immediately based on operation type and inputs according to the following table:

op_sel	Operation	Output Result
2'b00	Increment	data_out = data_in + 1
2'b01	Decrement	data_out = data_in - 1
2'b10	Bitwise NOT	data_out = ~data_in
2'b11	Left Shift	data_out = data_in << 1
default	Pass-through	data_out = data_in

Motivation:

Building a golden model using SV is not always an easy task. Even after building a golden model, making sure it's working as a perfect golden model needs a verification. In such cases, the methodology is to build a golden model using a dedicated tool for modeling (such as Matlab, or Python). Using this new model, we can get the input stimuli and the corresponding expected golden outputs. Further more, these input stimuli and the corresponding expected golden outputs are placed into files, later these files are used for verification of DUT, or even a SV golden model. This is called "**File-based test**".

Objective:

A **File-based test** is a test in which the **golden model is not a SV task/function or program, instead you are given one or more files (No SV randomization)**. Some of these files are for input stimuli of the DUT, others are the expected golden output corresponds to these input stimuli.

The **File-based test** works as following: First, open the given files and read the input stimuli files and the expected output files and store their contents. Second, process the input stimuli (if required) and input it to the DUT. Finally, collect the output and compare it vs the expected golden outputs which you have just stored after reading their files.

We need to test the **multi_op_processor** combinational DUT a **File-based test** using proper data types and routines. This means **you will be given file-based model**. Simply, our test flow will be as following:

- 1- Read the file-based golden model files to have the given stimuli and the expected output.
- 2- Test the DUT: Input the given stimuli to the DUT and test DUT output vs the the expected given output.

Provided files:

- inputs.txt — It contains 200 input line. Each line is a **10-bit binary** value formatted as {data_in[7:0], op_sel[1:0]}. The upper 8 bits are data_in, the lower 2 bits are op_sel.
- expected.txt — It contains 200 expected output line. Each line is the corresponding **8-bit binary** data_out that a correct DUT must produce for the input on the *same line number* of inputs.txt.

So for any line i: data_in = inputs[i][9:2], op_sel = inputs[i][1:0], and a correct DUT must give data_out == expected[i].

Steps:

- Declare Dynamic array of proper datatype (Choose the most appropriate one in your opinion and mention why you chose it) to store the stimulus for the design.
- Queue to Collect the output values from the design.
- Associative array holds the expected values from the golden model and storing them with index type int (the index represents the stimulus number).
- Declare the following tasks or functions with the following specs:
 - **drive_stim ()**: Task which iterates through previously prepared stimulus with the dynamic array and send the stimulus to DUT.
 - **collect_output_data (<data from design outputs>)**: To collect the data have been read from the design and store it with the queue.
 - **check_results (<Expected data>, <Actual data>)**: Takes the expected data stored in the Associative array previously prepared in golden model **and** the actual output data from the design stored in the

queue **and** iterates through them and makes the checks with proper display statements to indicate whether the data is successful as expected or not.

- **read_bin_file(<File_name>, <Width>, <Queue>)**: Clears the <Queue> (passed by reference), reads binary tokens from the specified <File name>, verifies the expected bit width (<Width>), dynamically pushes valid values into the queue, and terminates if a fatal error on file access, parsing, or configuration (width) mismatches.
- **load_file_stimulus()**: Reads input stimulus and golden reference files using **read_bin_file**, allocates stimulus arrays based on file length, unpacks each 10-bit input token into data and operation fields, and stores the parsed golden values in an associative array for later verification.

Helpful Steps:

The expected flow you will walk through is to

1. Prepare the stimulus by reading the file.
2. Acquire the expected output by reading the file.
3. Driving the stimulus to the design and acquire its output both using **drive_stim ()**, then compare it to the expected output using **check_results**.

Use the tasks and functions described above to perform all the steps mentioned to automate the process

Deliverables:

You have to deliver

- **Zip Folder** contains:
 - The implemented **.sv** files and the **design files** and **run.do** file.
- **Separated PDF** contains:
 - Snippets for the implemented code.
 - Snippet for the waveform shows the values for created variables
 - Snippets for the logs show the all printed values.
 - Snippet for Do file you have used for automating the process.
 - In the PDF, include snippets of the file-reading code, the per-vector check log, and the final summary line.

The delivered zip file must be named like **your_name_Assignment_2_Extra.rar** for example:

Hassan_Khaled_Assignment_2_Extra.rar also the PDF file **Hassan_Khaled_Assignment_2_Extra.pdf**

Good Luck